	THE UNIVERSITY OF SHEFFIELD **School of Electrical and Electronic Engineering** **Individual Project** **Interim report**		
Student Name	Yuqi Guo (ID: 230184273)		
Project Title	Hardware Acceleration for Cluster Fault Tolerance in Hybrid CMOS/non-CMOS Memories		
Supervisor	Mr. Neil Powell	**Second Marker**	

1. Background, Aims and Objects

1.1 Background

The Residue Number System (RNS) was first formalized by Garner as a non-weighted numeral representation enabling parallel arithmetic. Its fault-tolerant extension—Redundant RNS (RRNS)—was later established by Barsi and Maestrini, who demonstrated that adding redundant moduli enables detection and correction of multiple residue errors through consistency checks [4]. Early RRNS applications focused on digital signal processing and communication systems, where its carry-free computation offered speed and error resilience advantages [5].

A pivotal shift toward memory systems occurred when Haron and Hamdioui proposed RRNS specifically for cluster fault tolerance in high-density hybrid CMOS/non-CMOS memories [1]. Their work highlighted RRNS's unique capability to correct spatially correlated multi-bit errors—a challenge for conventional ECCs like Reed-Solomon codes. However, the original RRNS implementation incurred significant storage overhead due to large redundant moduli. Subsequent efforts sought hardware-efficient variants: Goh and Siddiqi analysed multiple error correction limits in RRNS [2], while Kumar et al. demonstrated an FPGA-based RRNS decoder for nanoscale memories [3], confirming the feasibility of hardware acceleration.

Recent research has prioritized reducing complexity without sacrificing reliability. Kalantar et al. introduced lightweight RRNS codes tailored for non-volatile memories [6], and Wang & Azarderakhsh enhanced decoding robustness via maximum-likelihood methods—directly motivating this project's adoption of MLD [7]. A comprehensive survey by Xiao & Hu confirms that "low-overhead RRNS design remains an open challenge" in next-generation memory systems [8]. Building on these advances, this work proposes two novel RRNS variants—2NRM-RRNS and 3NRM-RRNS—that achieve competitive fault tolerance with minimal storage cost, addressing the key limitation identified in [1].

1.2 Aims

- **Algorithm Validation:** Quantitatively evaluate the error correction capability and performance advantages of 3NRM-RRNS and 2NRM-RRNS codes through MATLAB simulation, with specific focus on cluster fault tolerance capability compared to C-RRNS and RS codes
- **Hardware Implementation:** Design and implement efficient encoder/decoder architectures for all four coding schemes (RS, C-RRNS, 3NRM-RRNS, 2NRM-RRNS) using hardware description languages
- **Performance Benchmarking:** Conduct comprehensive performance analysis including resource utilization, timing characteristics, and power consumption on FPGA platforms
- **Practical Validation:** Demonstrate the practical feasibility of the proposed RRNS variants for real-world memory protection applications

Expected Contributions:

- Comprehensive performance characterization of novel RRNS variants under realistic cluster fault models.
- Open-source implementations of optimized RRNS encoder/decoder architectures
- Practical validation of RRNS efficacy for next-generation memory protection
- Benchmark data comparing RRNS approaches with established techniques

1.3 Specifications

Simulation Framework Specifications:

- Implement four coding schemes: Reed-Solomon (RS-CODE), C-RRNS, 3NRM-RRNS, and 2NRM-RRNS with parameters matching [1]
- **Fault Injection:** Implement cluster fault models with varying sizes (1~20 bits clusters) and injection rates (1-10% of memory capacity)
- **Performance Metrics:**
  - Bit error rate (BER) vs. fault rate characteristics
  - Decoding latency measurements under different fault conditions
  - Memory overhead analysis (code rate efficiency)
- **Validation:** Reproduce key results from [1] to verify implementation correctness

Hardware Implementation Specifications:

- **Architecture Design:** Develop parameterizable encoder/decoder architectures supporting all four coding schemes
- **Performance Targets:**
  - Maximum operating frequency ≥ 100 MHz
  - Resource utilization optimization for each coding scheme
  - Latency measurements for encoding/decoding operations
- **Functional Verification:**
  - RTL simulation with comprehensive testbenches
  - FPGA-in-the-loop validation with realistic fault patterns
  - Error correction capability verification for specified t values

Process Management：

- Maintain a logbook documenting all development processes, experimental setups, and results.
- Produce a comprehensive report and prepare a presentation summarising the methodology, experimental results, and key contributions of the project.

2. Theory and Literature Review

2.1 RRNS

2.1.1 RRNS Core Principles

Redundant Residue Number Systems (RRNS) represent an advanced error correction coding technique based on the mathematical foundation of residue number systems. Unlike conventional coding approaches, RRNS employs a modular arithmetic framework that provides inherent advantages for fault tolerance applications, particularly in handling cluster errors in memory systems.

The fundamental principle of RRNS involves representing numerical data through a set of residues obtained by modulo operations with carefully selected moduli. This representation creates a redundant coding structure where the original information can be recovered even when some residues contain errors, making it particularly suitable for applications requiring high reliability.

2.1.2 Mathematical Foundation

The RRNS algorithm operates on the principle of Chinese Remainder Theorem (CRT), utilizing a set of pairwise relatively prime positive integers as moduli. The mathematical foundation ensures that any number within the legitimate range can be uniquely represented by its residues modulo the chosen moduli set, providing the basis for both encoding and error correction capabilities.

2.1.2.1 Moduli Selection and Configuration

The encoding process begins with the careful selection of moduli sets that satisfy three critical conditions:

Moduli Selection Rules:

[1]. **Pairwise coprimality:** $\gcd(m_i, m_j)=1$ for all $i \neq j$
[2]. **Strict ordering**: $m_1 < m_2 < \cdots < m_n$
[3]. **Dynamic range sufficiency**: $\prod_{i=1}^{k} m_i > X_{max}$

The moduli set is divided into two distinct subsets:

- **Non-redundant moduli (m_a)**: k moduli for data representation
- **Redundant moduli (m_b)**: (n-k) moduli for error correction

2.1.3 Encoding Algorithm

The encoding process transforms input data X into an n-tuple of residues through modular arithmetic operations: **Encoding Formula:**

$x_1 = |X|_{m1}, x_2 = |X|_{m2}, \ldots, x_n = |X|_{mn}$

2.1.4 Mixed-Radix Conversion (MRC) Decoding Algorithm

2.1.4.1 MRC Mathematical Foundation

Mixed-Radix Conversion provides an efficient method for reconstructing the original number from its residue representation. The MRC algorithm employs a systematic approach that utilizes mixed radix digits to recover the original value while enabling error detection and correction capabilities.

MRC Reconstruction Formula:

Mixed-Radix Conversion provides an efficient method for reconstructing the original number from its residue representation. The MRC algorithm employs a systematic approach that utilizes mixed radix digits to recover the original value while enabling error detection and correction capabilities.

MRC Reconstruction Formula:

$X_r = \sum v_s \times w_s$

where:

- v_s are the mixed-radix digits
- w_s are the weight coefficients

2.1.4.2 MRC Computational Procedure

The MRC algorithm proceeds through a sequential computation of mixed-radix digits:

Step 1: Initial Digit Calculation

$v_1 = x_1$

Step 2: Sequential Digit Computation

$v_s = |((x_s - v_1) \times m_{1s}^{-1} - v_2 \times m_{2s}^{-1} - ... - v_{s-1}) \times m_{(s-1)s}^{-1}|_{ms}$

where $m_{(s-h)s}^{-1}$ represents the multiplicative inverse satisfying $|m_{(s-h)} \times m_{(s-h)s}^{-1}|_{ms} = 1$

Step 3: Weight Coefficient Calculation

$w_1 = 1$
$w_s = \prod m_i$ for i=1 to s-1

2.1.5 Error Detection and Correction Mechanism

2.1.5.1 Validity Checking

The decoding process begins with validity verification through comparison with the legitimate range M_a:

Validation Criteria:

- If $X_n \leq M_a$: Codeword valid, no errors detected
- If $X_n > M_a$: Errors detected, correction required

2.1.5.2 Error Correction Procedure

When errors are detected, the system employs a trial-and-error approach:

Correction Algorithm:

[1]. Discard t residues (where t is error correction capability)
[2]. Compute X_2 from remaining z = (n-t) residues
[3]. Compare X_2 with product of remaining moduli M_2
[4]. Repeat for maximum C_t^n combinations

Success Condition:

The correct data is recovered when any $X_2 \leq M_2$, indicating successful error correction.

2.2 2NRM-RRNS & 3NRM-RRNS

The paper [1] introduces two optimized RRNS variants:

2.2.1 NRM-RRNS: Three Non-Redundant Moduli RRNS

The 3NRM-RRNS code employs the same non-redundant moduli set as C-RRNS: {64, 63, 65\} (i.e., {2^6, 2^6 - 1, 2^6 + 1}). Nevertheless, the unrestricted redundant moduli mb for 3NRM-RRNS are based on integers with smaller value that of

- Uses moduli set {64,63,65} for data representation
- Implements smaller redundant moduli {31,29,23,19,17,11}
- Maintains error correction capability t ≤ 3

2.2.2 2NRM-RRNS: Two Non-Redundant Moduli RRNS

The 2NRM-RRNS code uses two non-redundant moduli: $\{2^f + 1, 2^f\}$ with f = 8, i.e., {257, 256}. instead of three as in C-RRNS and 3NRM-RRNS, four arbitrary integers with smaller values than that of ma are selected for unrestricted moduli

- Employs moduli set {257,256} for compact representation
- Utilizes redundant moduli {61,59,55,53}
- Provides error correction capability $t \leq 2$

2.2.3 Performance Advantages

The optimized RRNS variants demonstrate significant improvements:

Efficiency:

- **2NRM-RRNS**: 41-bit codeword, correcting up to 41.5%-bit errors
- **3NRM-RRNS**: 48-bit codeword, 39.6% correction capability
- **RS Code**: 48-bit codeword, 33.3% correction capability
- **C-RRNS**: 61-bit codeword, 31.1% correction capability

2.3 Maximum Likelihood Decoding (MLD) And Hamming Distance Calculation

2.3.1. Maximum Likelihood Decoding (MLD) Algorithm

2.3.1.1 Fundamental Concept

Maximum Likelihood Decoding is a sophisticated error correction technique employed in modified RRNS codes to resolve decoding ambiguities that arise when using non-conventional moduli sets. The algorithm operates on the principle of selecting the most probable valid codeword from multiple potential candidates based on their similarity to the received codeword.

2.3.1.2 MLD in Modified RRNS Context

In the context of 2NRM-RRNS and 3NRM-RRNS codes, MLD addresses the violation of the traditional RRNS rule requiring redundant moduli to be larger than non-redundant moduli. This violation breaks the uniqueness guarantee of the Chinese Remainder Theorem, creating ambiguity during decoding—i.e., multiple candidate values may satisfy the validity condition $X \leqslant M_a$. To resolve this, MLD selects the most likely codeword based on Hamming distance.

MLD Implementation Process:

[1]. **Candidate Generation**: Identify all potential valid data values within the legitimate range
[2]. **Residue Reconstruction**: For each candidate, compute the expected residue values using modulo operations
[3]. **Similarity Assessment**: Compare reconstructed residues with actual received residues
[4]. **Optimal Selection**: Choose the candidate with minimum Hamming distance to the received codeword

2.1.2.3 Mathematical Formulation

The MLD algorithm can be formally expressed as:

$$X_{correct} = \text{argmin}_\{X \in \text{Valid_Set}\}\ d_H(R_{received}, R_{expected}(X))$$

where:

- $X_{correct}$ is the selected valid data
- Valid_Set contains all potential valid data values
- d_H represents Hamming distance function
- $R_{received}$ is the received residue vector
- $R_{expected}(X)$ is the expected residue vector for candidate X

2.3.2. Hamming Distance Calculation Methodology

2.3.2.1 Definition and Significance

Hamming distance serves as the fundamental metric in MLD algorithms [1] [2], quantifying the difference between two sequences of equal length. In RRNS applications, it measures the dissimilarity between residue vectors, providing a crucial criterion for error detection and correction.

Formal Definition:

The Hamming distance between two vectors A and B of equal length n is defined as:

$d_H(A, B) = \sum_{i=1}^n \delta(a_i, b_i)$

where $\delta(a_i, b_i) = 1$ if $a_i \neq b_i$, and 0 otherwise.

2.3.2.2 Application in RRNS Decoding

In RRNS systems, Hamming distance calculation operates on residue representations:

Residue-Level Distance Computation:

$d_H(R, R')$ = number of positions where $r_i \neq r'_i$

where $R = \{r_1, r_2, ..., r_n\}$ represents the received residue vector and $R' = \{r'_1, r'_2, ..., r'_n\}$ represents the expected residue vector.

2.3.3. Integrated MLD-Hamming Distance Workflow

3.1 Complete Decoding Process

The integration of MLD with Hamming distance calculation creates a comprehensive decoding framework:

Step 1: Initial Decoding Attempt

- Decode using standard RRNS procedures
- Check if result falls within legitimate range $[0, M_a]$

Step 2: Ambiguity Detection

- Identify multiple valid candidates when standard decoding fails or produces ambiguous results
- Generate all potential valid data values within the legitimate range

Step 3: Candidate Evaluation For each valid candidate $X_{candidate}$:

[1]. Compute expected residues: $R_{expected} = \{|X_{candidate}|_{m1}, |X_{candidate}|_{m2}, ..., |X_{candidate}|_{mn}\}$
[2]. Calculate Hamming distance: $d_H(R_{received}, R_{expected})$
[3]. Record distance measurement

Step 4: Optimal Selection

- Select candidate with minimum Hamming distance

- If multiple candidates share minimum distance, apply secondary criteria (if defined)

3. Technical Progress

The following work has been completed so far:

- In-depth research on RS-CODE, C-RRNS, 3NRM-RRNS, 2NRM-RRNS algorithms
- In-depth research on MRC, MLD algorithms, Hamming distance and their application in RRNS decoding
- Architecture design for MATLAB simulation programs
- MATLAB simulation program coding and debugging
- MATLAB simulation program execution and data analysis

Table 1 Table 1. Moduli sets used in simulation for each ECC scheme.

ECC Types	Non-Redundant, m_k	Redundant, $m_{(n-k)}$
C-RRNS	$\{2^f, 2^f - 1, 2^f + 1\}$ f=6 = {64,63,65}	{67,71,73,79,83,89}
3NRM-RRNS	$\{2^f, 2^f - 1, 2^f + 1\}$ f=6 = {64,63,65}	{31,29,23,19,17,11}
2NRM-RRNS	$\{2^f + 1, 2^f\}$ f=8 = {257,256}	{61,59,55,53}

Reed-Solomon (RS) code parameters: n = 12, k = 4, t = 4, symbol size m = 4 bits.

fault_rates = 0:0.005:0.15

Number of Monte Carlo trials: 1000

Test data words: Fifteen random integers between 0 and 65,535.

3.1 Simulation Program Architecture

The top architecture of the simulation program is illustrated as following figure 1:

Top-level flow: test cases generation → encoding → fault injection → decoding → analysis

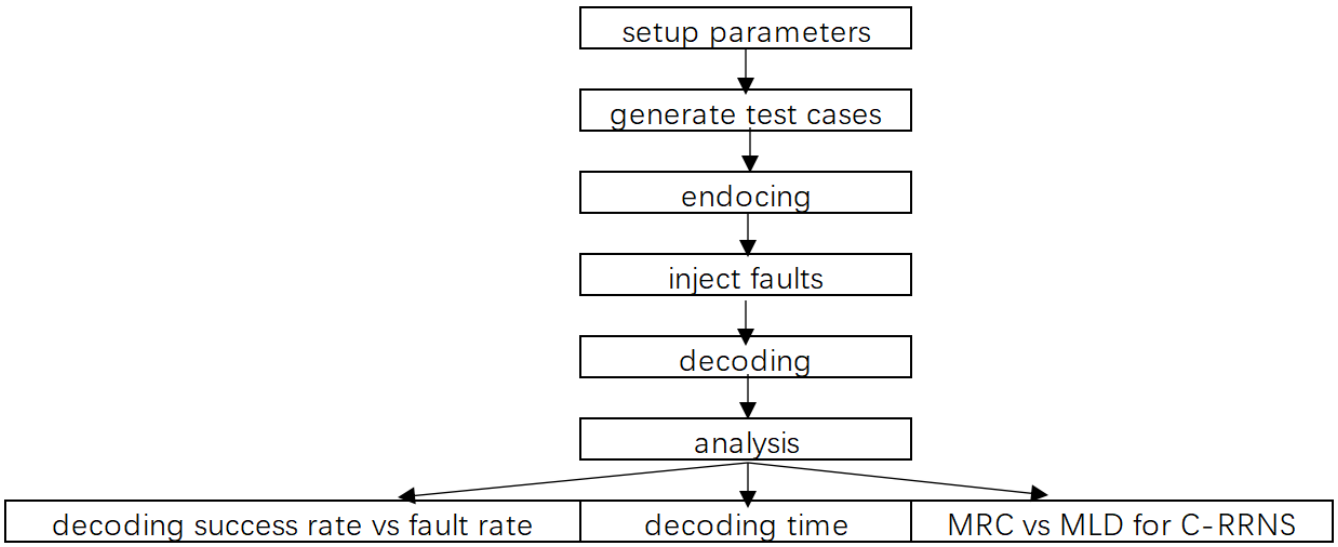


Figure 1 architecture of simulation program

Table 2 simulation program function description

Function	Description
setup_parameters	Setup all the parameters of the simulation program
generate_test_cases	create all the test cases for all coding schemes: C-RRNS,3NRM-RRNS,2NRM-RRNS, RS-CODE
encoding	use the scheme in the test case to coding the data word
inject faults	inject the error bits error according to the fault rate
decoding	use the decoder to decode the interrupted data word and record the success rate
Analysis	post analysis the data collected in the simulation, include the decoding success rate vs fault rate, decoding time consumption and the performance of C-RRNS using MCR or MLD decoder algorithm

3.2 Simulation Results and Analysis

Table 3 shows the total number of bits that represent the codeword for each code. RS, C-RRNS,3NRM-RRNS, and 2NRM-RRNS are represented by 48, 61, 48, and 41 bits. C-RRNS, 3NRM-RRNS, and 2NRMRRNS produce bit length difference of 27.1% longer, equal, and 14.6% shorter, respectively, when compare to RS. 3NRM-RRNS and 2NRM-RRNS are shorter by 21.3% and 32.8%, each, when compared to C-RRNS.[1]

Table 3 16 bits word Reed Solomon and Redundant Residue Number Systems variant codes

ECC Type	Number of bits			Difference (%)	
	Dataword	Check word	Codeword	Compare with RS	Compare with C-RRNS
RS	4,4,4,4	4,4,4,4,4,4,4,4	48	-	
C-RRNS	6,6,7	7,7,7,7,7,7	61	+27.1	-
3NRM-RRNS	6,6,7	5,5,5,5,5,4	48	0	-21.3
2NRM-RRNS	9,8	6,6,6,6	41	-14.6	-32.8

According to the simulation test results, the innovative algorithms 3NRM-RRNS and 2NRM-RRNS proposed in Paper [1], combined with MLD and Hamming distance, demonstrate relatively high data recovery capability in the presence of bit errors. The recovery capabilities of various algorithms under different error conditions are shown in the table 4 below.

Table 4 Maximum fault rate achieving 100% decoding success across 1000 trials.

Redundancy Number System	Maximum allowable fault rate for 100% data recovery
C-RRNS	7% (t = 3)
3NRM-RRNS	7% (t =3)
2NRM-RRNS	6% (t = 2)
RS-CODE	9% (t = 4)

3.2.1 Decoding Performance Comparison (C-RRNS: MRC)

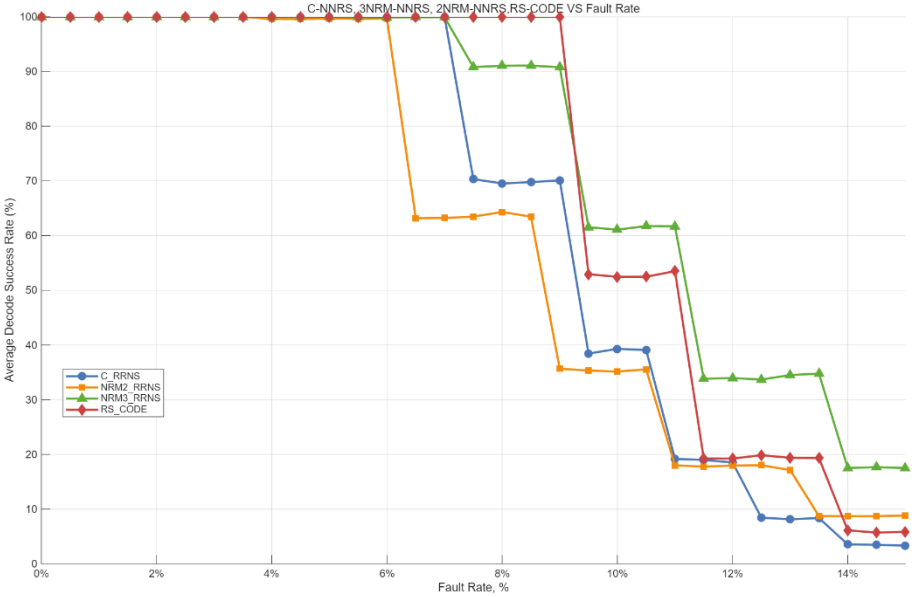


Figure 2 Average decoding success rate vs fault rate (CRRNS: MRC)

Decoding Success Rate vs Fault Rate (C-RRNS: MRC) C-RRNS uses a self-implemented MRC decoding algorithm, 3NRM-RRNS and 2NRM-RRNS use a self-implemented MLD decoding algorithm, and RS-CODE utilizes the built-in RS algorithm in MATLAB. The simulation results are summarized in figure 2.

Key Finds

[1]. **RS_CODE Exhibits Superior Robustness at High Fault Rates**
The RS_CODE (Reed-Solomon Code) maintains a 100% decoding success rate up to approximately **9.5% fault rate**, after which it rapidly degrades. However, even at 14%, it still achieves ~6–8% success, indicating strong resilience in high-error environments.

[2]. **C_RRNS Performs Well at Low Fault Rates but Degrades Rapidly**
C_RRNS shows excellent performance at low fault rates (<6%), with 100% success. However, its performance drops sharply beyond 6%, falling below 20% by 12%. This suggests that while effective for minor faults, it is vulnerable to moderate-to-high error levels due to limited redundancy and voting-based correction mechanisms.

[3]. **NRM2_RRNS Offers Moderate Fault Tolerance**
RM2_RRNS performs better than C_RRNS in mid-range fault rates (6–10%) but starts failing earlier than RS_CODE. It stabilizes around 35–37% success rate between 8–11%, indicating partial error correction capability but insufficient for severe faults.

[4]. **NRM3_RRNS Shows Stronger Resilience Than NRM2_RRNS**
with higher redundancy, NRM3_RRNS outperforms NRM2_RRNS across most fault rates. It sustains over 60% success until ~11%, then gradually declines to ~18% at 14%. This reflects the benefit of increased redundancy in RRNS systems.

[5]. **All Codes Fail Gradually Beyond 12% Fault Rate**
At fault rates exceeding 12%, all codes show significant degradation. However, **RS_CODE remains superior** in this regime, followed by NRM3_RRNS, then NRM2_RRNS and C_RRNS.

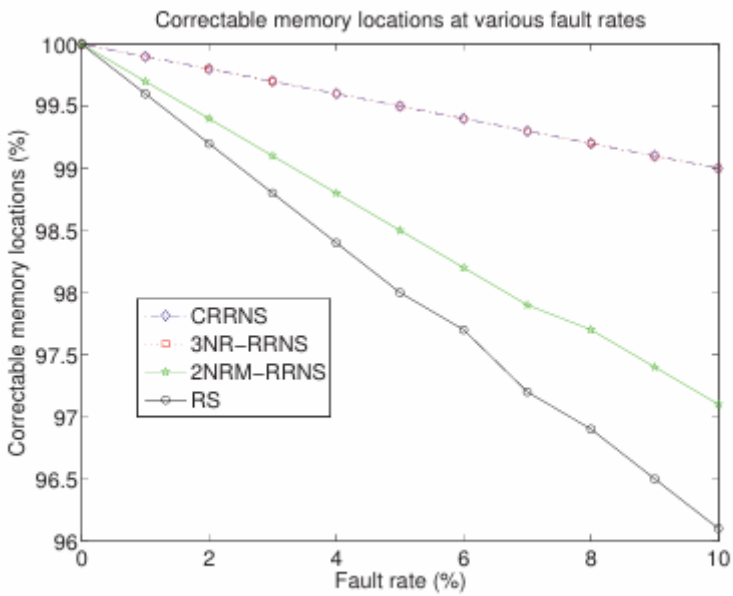
[6]. **Trade-off Between Redundancy and Performance**
The results highlight a clear trade-off:

- **RS_CODE** uses global parity and provides robust correction.

- **RRNS variants** rely on modular arithmetic and subset voting, offering fast encoding/decoding but lower tolerance under high fault conditions.

- The simulation reveals that **RS_CODE is the most reliable option under high fault rates**, whereas **C_RRNS is suitable only for low-noise environments**. Among RRNS schemes, **NRM3_RRNS offers the best balance of redundancy and fault tolerance**, making it a promising candidate for applications requiring moderate reliability with lower computational overhead.

It is worth noting that the correctability curve presented in Reference [1] does not exhibit the expected 100% plateau region at low fault rates (Figure 3). According to the algebraic properties of the Redundant Residue Number System (RRNS), when the number of redundant moduli is sufficient to correct up to errors, the system should achieve perfect error correction under low fault rates—specifically, when the probability of encountering more than errors is negligible. The simulation results in this work (see Figure 2) clearly demonstrate such a plateau, consistent with theoretical expectations. The discrepancy with the results reported in the original paper may stem from differences in decoding strategy, fault injection model, or the definition of the performance metric.



(a) Correctable memory locations at various fault rates

Figure 3 Simulation result in reference [1]

3.2.2 Decoding Performance Comparison (C-RRNS: MLD)

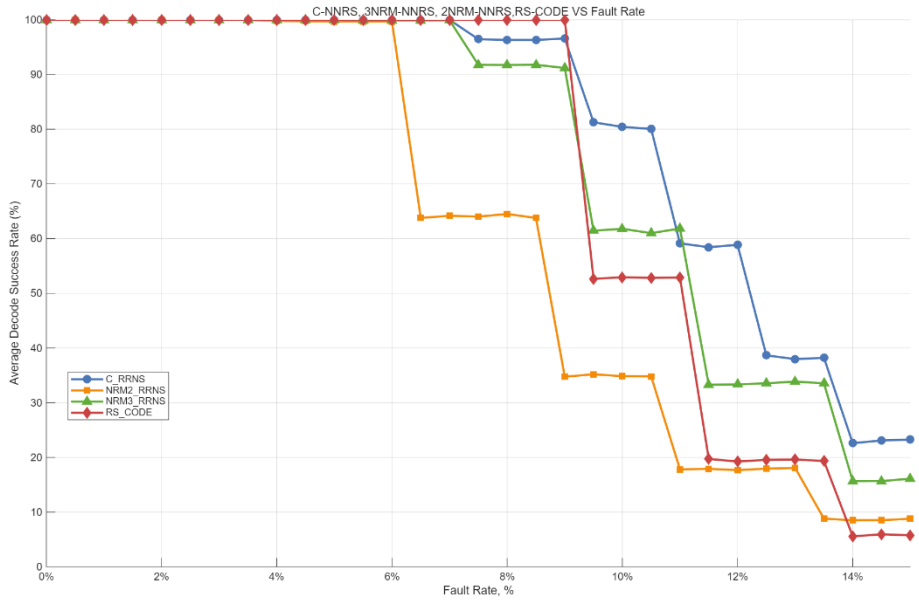


Figure 4 Average decoding success rate vs fault rate (C-RRNS: MLD)

C-RRNS uses the same MLD decoding algorithm as 3NRM-RRNS and 2NRM-RRNS, the simulation results are summarized in figure 4.

Key Observations:

Impact of MLD on C-RRNS Significant Improvement in High-Fault Regime

Compared to the original MRC-based C-RRNS, the version employing **Maximum Likelihood Decoding (MLD)** demonstrates a **marked improvement in decoding success rate at fault rates above 10%**. While the original C-RRNS drops below 20% at 12%, the MLD-enhanced version maintains ~38–40% success up to 12%, and remains above 20% even at 14%.

Preservation of High Performance at Low Fault Rates

At fault rates below 6%, both MRC and MLD versions of C-RRNS achieve **100% success**, indicating that MLD does not degrade performance under low-error conditions.

Extended Error-Correction Capability

The MLD approach allows C-RRNS to better distinguish between correct and erroneous residues by evaluating likelihoods across all possible values, rather than relying solely on majority voting. This leads to **more robust decisions in high-noise environments**.

Closer Performance to NRM3_RRNS and RS_CODE

With MLD, C-RRNS outperforms NRM2_RRNS and approaches the resilience of NRM3_RRNS in the 10–14% fault range, narrowing the gap with more advanced RRNS schemes.

Trade-off: Complexity vs. Reliability

Although MLD improves reliability, it comes at the cost of increased computational complexity due to exhaustive search or likelihood computation. However, this trade-off may be acceptable for applications requiring higher fault tolerance.

3.2.3 Decoding Latency Comparison (C-RRNS: MRC)

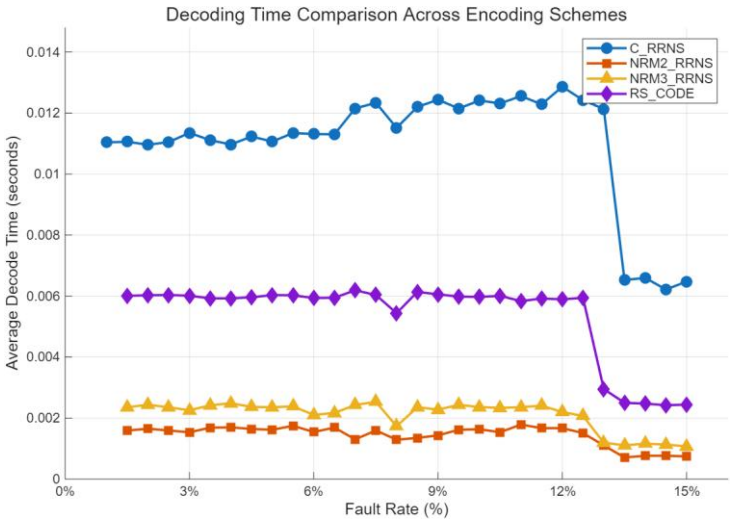


Figure 5 Average decoding latency (in MATLAB seconds) for C-RRNS using MRC decoder.

The decoding time for **C_RRNS** is significantly higher than other schemes (refer to figure 5), especially at low fault rates, due to its complex subset voting and MLD-based decoding. In contrast, **NRM2_RRNS**, **NRM3_RRNS**, and **RS_CODE** exhibit much faster and more stable decoding times, with RS_CODE showing minimal variation across fault rates. This highlights a trade-off between decoding speed and error correction capability, where C_RRNS sacrifices efficiency for improved fault tolerance.

3.2.4 Decoding Time Comparison (C-RRNS: MLD)

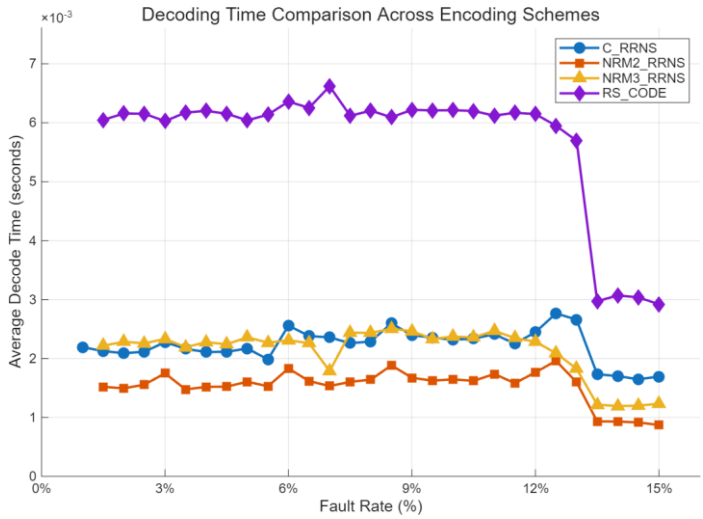


Figure 6 Average decoding latency (in MATLAB seconds) for C-RRNS using MLD decoder.

The adoption of **Maximum Likelihood Decoding (MLD)** in C-RRNS results in a **significant reduction in average decoding time** (refer to figure 6), dropping from ~10–12 ms (in earlier versions) to around **2–3 ms** across most fault rates. This improvement is primarily due to **elimination of redundant subset enumeration and majority voting**, which were computationally expensive in the original MRC-based approach. Instead, MLD employs a more direct likelihood-based decision process that scales better with fault rate, leading to faster and more consistent performance — especially noticeable at low-to-moderate error levels.

3.2.5 Summary of Findings

Feasibility: Simulation results confirm that the innovative 3NRM-RRNS and 2NRM-RRNS algorithms proposed in [1] are viable ECC schemes for hybrid CMOS/non-CMOS memories.

Fault Tolerance: Across various fault rates, all algorithms provide satisfactory redundancy protection; notably, 3NRM-RRNS offers comparable protection to C-RRNS.

Storage Efficiency: In terms of storage efficiency, 2NRM-RRNS stands out as the most efficient, requiring minimal storage space.

Recommendation: Considering both performance and storage efficiency, 2NRM-RRNS is the recommended ECC algorithm for hybrid CMOS/non-CMOS memory systems.

4. Project Management

4.1 Gantt Chart and Overall Progress

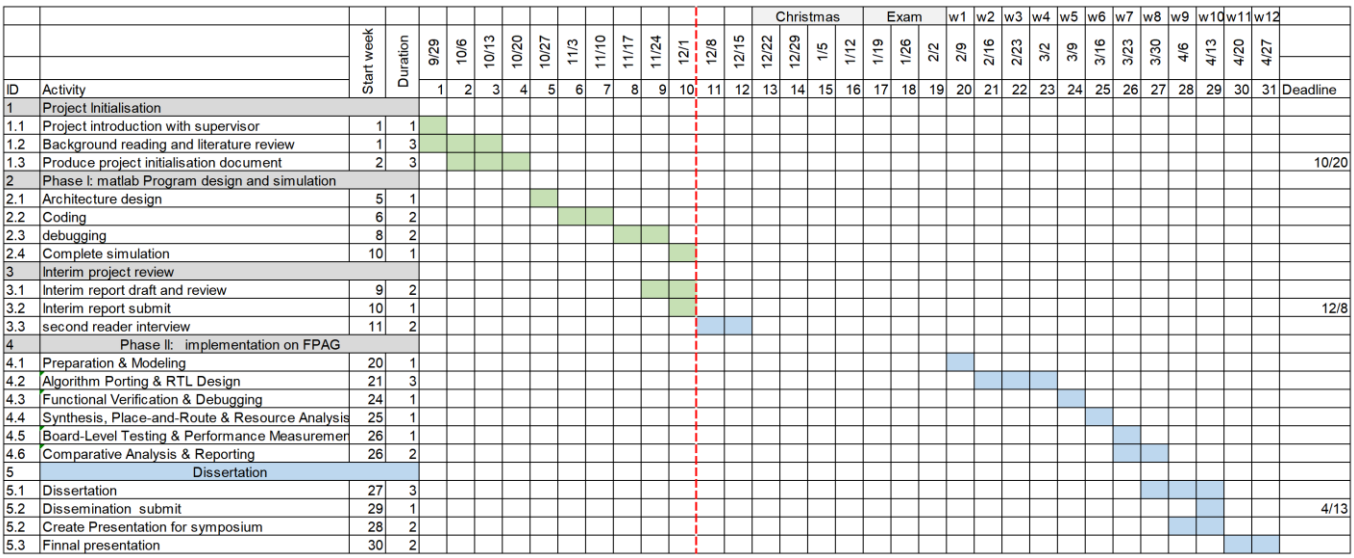


Figure 7 project plan Gantt chart

4.2 Work Completed

- MATLAB program design and coding, debugging
- MATLAB simulation to compare the performance of 4 scheme of algorithm: RS, C-RRNS, 3NRM-RRNS, 2NRM-NNRS
- Interim report

Detail information please refer to Figure 6, current date W11(red line in Figure 7).

4.3 Planned Work

- Implement parameterizable RRNS encoder/decoder on Xilinx Artix-7 FPGA (If MLD latency exceeds target, explore lookup-table optimization)
- Measure resource utilization (LUTs, FFs) and max clock frequency
- Dissertation preparation

please refer to the Gantt chart for detail project plan.

5. References:

[1] N. Z. Haron and S. Hamdioui, "Using RRNS Codes for Cluster Faults Tolerance in Hybrid Memories",*2009 24th IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems*, 2009, pp. 85-93. doi: 10.1109/DFT.2009.37.

[2] V. T. Goh and M. U. Siddiqi, "Multiple Error Detection and Correction based on Redundant Residue Number Systems", *IEEE Transactions on Communications*, vol. 56, no. 3, pp. 325–330, March 2008.

[3] A. Kumar et al., "FPGA-Based RRNS Decoder for Nanoscale Memory Systems," *IEEE Trans. Very Large Scale Integr. (TVLSI)*, vol. 30, no. 5, pp. 789–793, May 2022.

[4] F. Barsi and P. Maestrini, "Error Detection and Correction by Means of Redundant Residue Number Systems," *IEEE Trans. Comput.*, vol. C-23, no. 3, pp. 307–315, Mar. 1974.

[5] M. A. Soderstrand, W. K. Jenkins, G. A. Jullien, and F. J. Taylor, *Residue Number System Arithmetic: Modern Applications in Digital Signal Processing.* New York, NY, USA: IEEE Press, 1986.

[6] M. B. Kalantar, M. R. Ebrahimi, and A. Ejlali, "Efficient RRNS-Based Error Control Codes for Non-Volatile Memories," *IEEE Trans. Circuits Syst*. I, Reg. Papers, vol. 68, no. 5, pp. 1987–1998, May 2021.

[7] Y. Wang and M. H. Azarderakhsh, "Maximum-Likelihood Decoding for RRNS-Based Fault-Tolerant Systems," in *Proc. IEEE Int. Symp. Circuits Syst.* (ISCAS), 2022, pp. 2105–2109.

[8] L. Xiao and J. Hu, "A Survey on Residue Number System: Theory, Applications, and Challenges," *ACM Comput. Surv.*, vol. 55, no. 4, Art. no. 78, pp. 1–36, Apr. 2023.